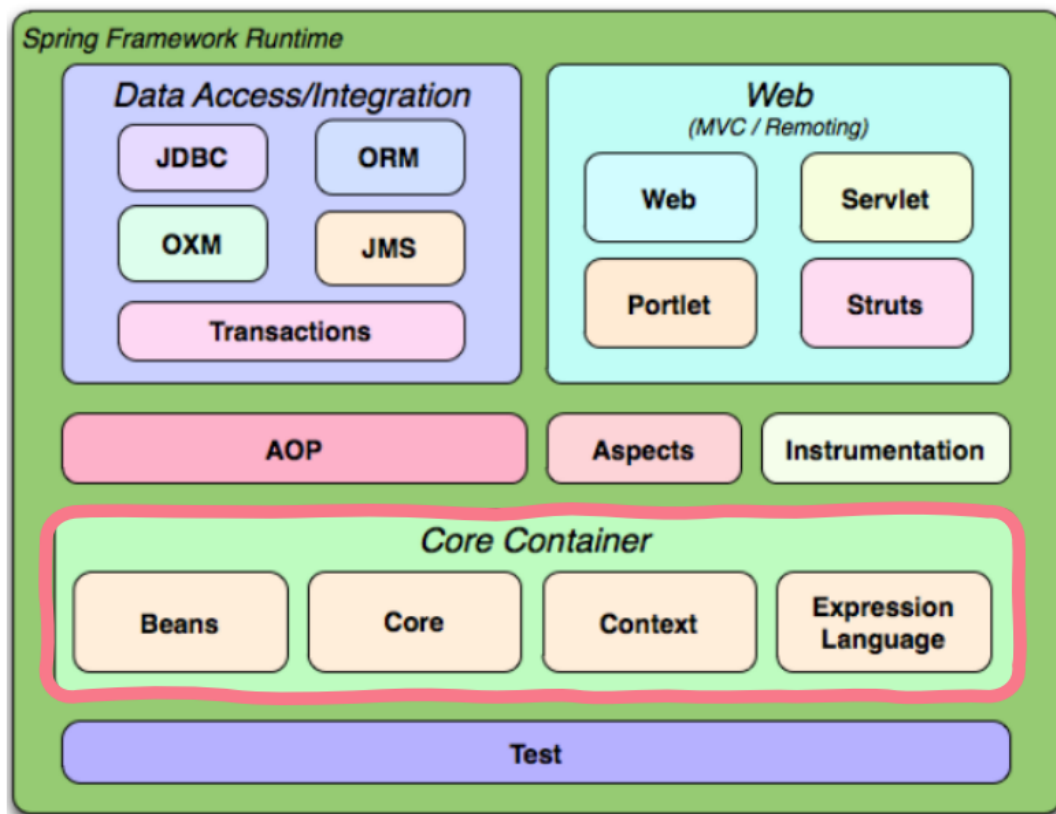


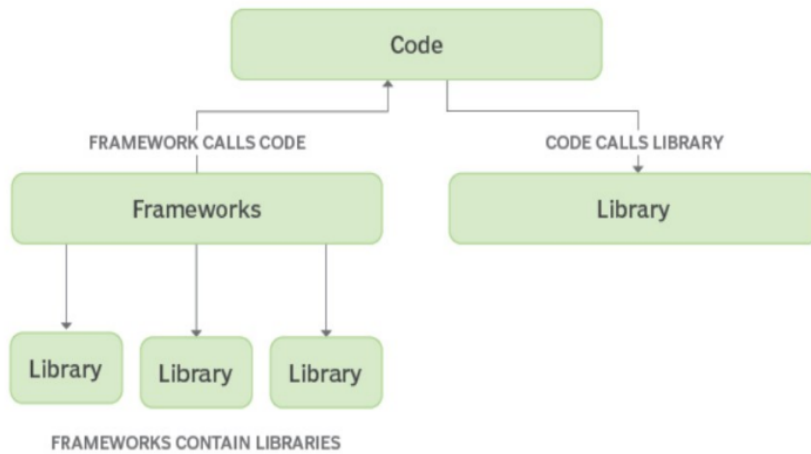
Lecture 12 -- Spring

• The Spring Framework

- Spring 框架简介
 - 开源、轻量级框架，帮助开发企业级应用
 - 解决早期 Java EE 的若干问题：
 - 用简单的 POJO（普通 Java 对象）替代繁重的 EJB
 - 通过依赖注入管理组件依赖关系
 - 支持使用轻量级容器（如 Tomcat）而非完整应用服务器（如 GlassFish）
- Spring 模块与核心容器
 - Spring 由大约 20 个模块组成（示意图所示）



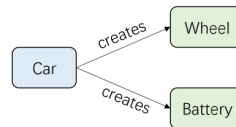
- Spring Core 容器为必要模块，其他模块可选



- 核心容器基于控制反转（IoC）与依赖注入（DI）
- **控制反转（IoC）与依赖注入 Dependency Injection（DI）**
 - 控制反转（IoC）：将程序中对象的控制权交给容器或框架管理
 - 传统代码调用库；IoC 则由框架控制程序流程并调用用户代码
 - 使用框架时，需要通过继承或插件方式将自定义行为插入框架的各个点，框架在适当时机调用这些代码
 - 依赖注入（DI）：Spring 中实现 IoC 的具体方式，用于注入和管理对象依赖关系
 - 无依赖注入：

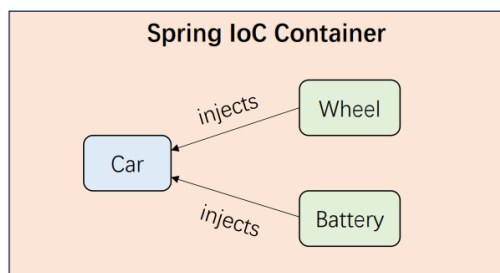
```

class Car {
    private Wheel wheel = new NepaliRubberWheel();
    private Battery battery = new ExcideBattery();
    // .....
}
  
```



- Car 对象负责自己创建依赖对象 Wheel 和 Battery
- 代码高度耦合（若 Battery 的构造器变化，Car 会出问题）
- 难以测试（例如难以对 Car 进行单元测试）
- 使用依赖注入：

- Spring IoC Container creates and injecting the dependencies (Wheel and Battery) at runtime.
- Injection can be done by **setter injection** or **constructor injection**.



```

class Car {
    private Wheel wheel;
    private Battery battery;

    public Car(Wheel wheel, Battery battery) {
        this.wheel = wheel;
        this.battery = battery;
    }

    void setWheel(Wheel wheel) {
        this.wheel = wheel;
    }

    void setBattery(Battery battery) {
        this.battery = battery;
    }
}
  
```

- Spring IoC 容器在运行时创建并注入依赖（如 Wheel 和 Battery）
- 注入方式可为构造器注入或 setter 注入

- 通过注入降低类之间的耦合，便于替换依赖与单元测试
- 依赖注入：将对象所需的实例变量由容器提供并赋值
 - 依赖注入（DI）：在 Spring 中实现 IoC 的方式
 - 用容器管理依赖后，业务代码只关注逻辑，不负责创建依赖对象
 - 由容器在外部注入对象的依赖，而不是对象内部自行创建依赖
 - 通过将对象的使用（由调用方）与对象的创建（由 IoC 容器）分离，实现低耦合的程序结构
- Spring 中的注解：
 - 业务逻辑示例：Car 与 Engine 为 POJO/Bean，Car 依赖 Engine（由容器注入）

```
@Component
public class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        System.out.println("Driving with " + engine.getName() + " engine.");
    }
}
```

- Business logics: Car and Engine (POJOs or Beans)
- Car depends on Engine

```
public class Engine {
    private String name;

    public Engine() {}

    public Engine(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void start() {
        System.out.println(name + " engine is starting.");
    }
}
```

- 使用 @Component（或类似注解）将类声明为可由容器管理的 Bean
 - @Component 用于自动检测并注册 Bean
 - Spring IoC 容器会：
 - 扫描应用中带 @Component 注解的类
 - 实例化这些类并注入其指定的依赖
 - 在需要时将该 Bean 注入到其他组件中
- 使用 @Autowired（或构造器/方法注入）让容器注入所需依赖
 - @Autowired 可用于 setter 方法、构造器等处
 - @Autowired 注解会隐式注入对象的依赖
 - 自动装配让 Spring 容器通过检查已配置的 Bean 自动解析并注入组件间的依赖关系

```

@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine("V8");
    }

}

```

A Java-based configuration class
(i.e., a class annotated with @Configuration)

- @Configuration
 - 标注的 Java 类即为一个配置类
 - 带 @Configuration 的类用于定义并实例化 Bean
- @ComponentScan
 - 与 @Configuration 配合使用，指定需要扫描的包路径以发现并注册 @Component 类

```

public class Engine {
    private String name;

    public Engine() {}

    public Engine(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void start() {
        System.out.println(name + " engine is starting.");
    }
}

@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine("V8");
    }

}

```

- Bean：由 Spring IoC 容器实例化、组装并管理的对象
- @Bean 注解用于配置类中的方法，用来创建并返回实际的 Bean 实例
- 使用 @Bean 可对 Bean 的创建进行更细粒度控制（例如在实例化时设置参数），与类级的 @Component 互补
- Spring IoC 容器：

```

public class Main {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        Car car = context.getBean(Car.class);
        car.drive();
    }
}

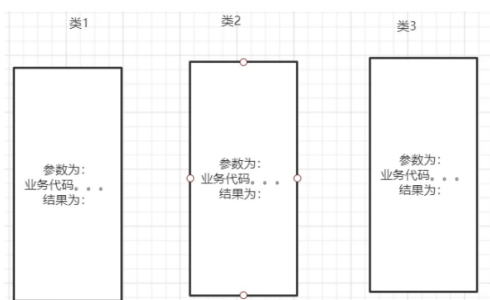
```

- 负责实例化、配置与组装对象/Bean（通过依赖注入），并管理它们的生命周期（即控制反转）
- 常用的容器接口为 ApplicationContext

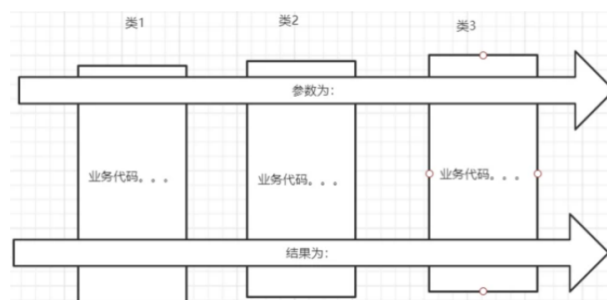
- 应用类与配置元数据结合后，创建并初始化 ApplicationContext，即得到一个完整配置且可执行的系统或应用

• Spring 切面编程（AOP）

- AOP（面向切面编程）：补充面向对象，便于分离横切关注点（将额外行为添加到现有代码而无需修改原有代码）
 - 横切关注点示例：日志记录、安全、事务管理、性能监控等
- Spring 提供 AOP 支持，可在框架层统一管理这些横切逻辑



Without AOP: business code and non-business code are tangled together



With AOP: business code and non-business code are **decoupled** and can be managed independently.

- 无 AOP：业务代码与非业务代码（如日志、安全）交织在一起，难以管理
- 有 AOP：将业务逻辑与非业务横切逻辑解耦，分别管理，代码更清晰且更易维护
- AOP 术语：
 - 切面（Aspect）：横切关注点的封装（如日志、安全、事务），在 Spring AOP 中通常用带 @Aspect 的类实现
 - 连接点（Join point）：程序执行中的某个点，在 Spring AOP 中通常表示方法的执行点
 - 通知（Advice）：切面在某个连接点执行的动作类型（例如 before、after、around 等）
 - 切入点表达式（Pointcut）：匹配连接点的谓词或表达式，指定哪些方法会被通知触发

Spring AOP Example

logBeforeV1() will be executed before getEmployeeById() during runtime

```
@Component
public class EmployeeManager {
    {
        public EmployeeDTO getEmployeeById(Integer employeeId) {
            System.out.println("Method getEmployeeById() called");
            return new EmployeeDTO();
        }
    }
}
```

Join point: Business logic

```
@Aspect
public class EmployeeCRUDAspect {
    {
        @Before("execution(* EmployeeManager.getEmployeeById(..))") //point-cut expression
        public void logBeforeV1(JoinPoint joinPoint) {
            System.out.println("EmployeeCRUDAspect.logBeforeV1() : " + joinPoint.getSignature().getName());
        }
    }
}
```

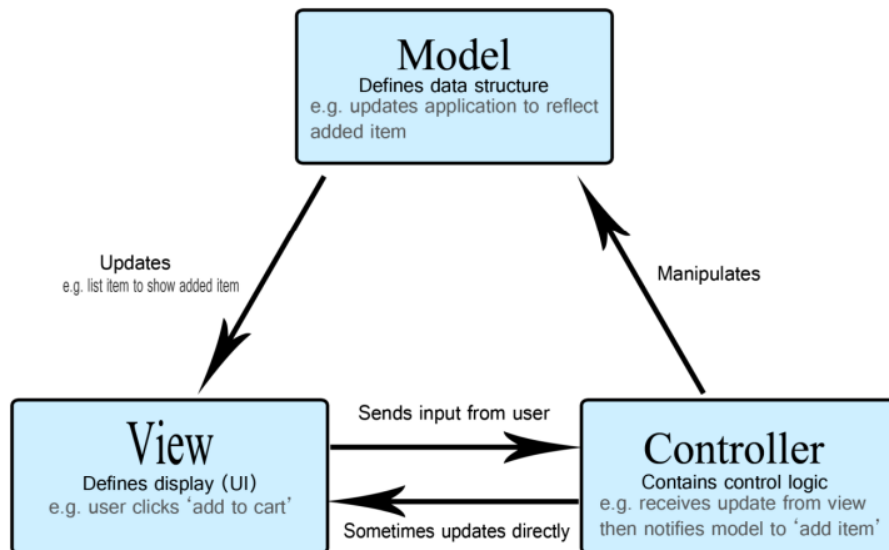
Aspect: Cross-cutting logic (logging)

Pointcut: expressions to match join-point methods

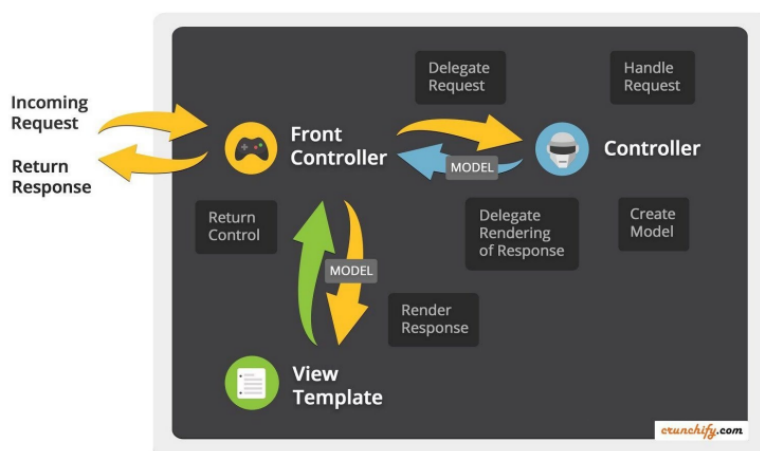
Before Advice: action taken at a join point

• Spring MVC (Web 框架)

- Spring MVC 是基于 Spring 核心构建的专用模块，提供依赖注入与控制反转等基础能力
- 在此基础上，Spring MVC 增强了处理 HTTP 请求与构建 Web 应用的能力
- MVC 模式将应用分为不同关注点（输入逻辑、业务逻辑、界面逻辑），有利于职责分离与可维护性
- MVC 设计模式：



- 模型（Model）：管理应用的数据、业务规则与状态（负责核心数据与业务处理）
 - 视图（View）：负责数据显示与用户界面呈现（将模型数据可视化）
 - 控制器（Controller）：接收用户输入并将其转化为对模型或视图的命令或更新（协调模型与视图的交互）
- Spring MVC 流程要点



- 客户端发送请求，先被前端控制器（DispatcherServlet）拦截
- 前端控制器将请求分发给合适的 Controller（控制器）
- Controller 处理请求并返回 Model 与 View 给前端控制器
- 前端控制器将渲染过程委派给返回的 View

- View 渲染 Model 数据并将响应返回给客户端

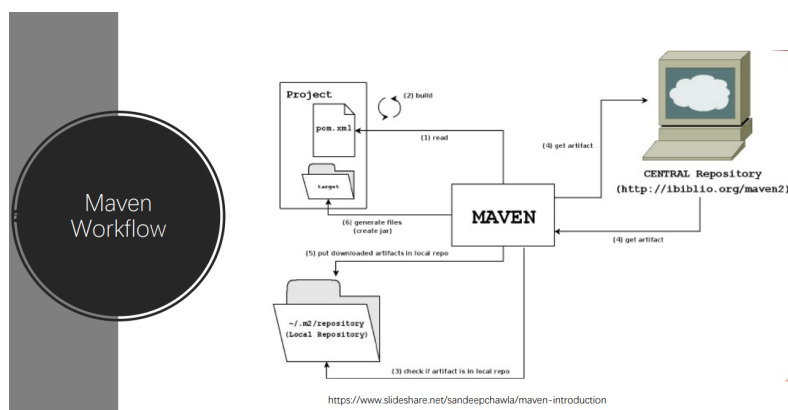
• Spring Boot

• Overview

- Spring 框架功能强大但配置繁琐，开发者需要手动完成许多重复配置
- Spring Boot 简化并自动化配置过程，加快 Spring 应用的创建与部署
- 可用来构建独立运行的应用，减少或几乎无需额外配置开销
- Spring Boot 指以一种方式引导 Spring 应用，使其包含运行完整应用所需的几乎所有内容。
- Spring Boot 的自动配置会根据你添加的 jar 依赖尝试自动配置你的 Spring 应用。
- Spring Boot 采取一种“有主见”的配置方式，指导你按其推荐的方法进行配置。

• 构建 MVC Web 应用

- 使用 Spring Boot
 - 使用 Spring Initializr 创建一个 Spring Boot 应用
 - 选择依赖（例如：Spring Web）
 - 完成
- 使用传统 Spring MVC
 - 下载并手动配置框架
 - 手动添加 Maven 依赖，例如：
 - spring-core / spring-context / spring-aop / spring-webmvc / spring-web
 - 需要各类配置，还有更多配置需求
- Maven 依赖



- POM 是“Project Object Model（项目对象模型）”的缩写。它是以 XML 形式表示的 Maven 项目，存放在名为 pom.xml 的文件中。
 - pom.xml 文件是 Maven 中项目配置的核心。
 - 它是一个单一的配置文件，包含构建项目所需的大部分信息和依赖，使你能够以期望的方式构建项目。
- spring-boot-starter-parent

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.
<modelVersion>4.0.0</modelVersion>
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.example</groupId>
<artifactId>SpringDemo</artifactId>
```

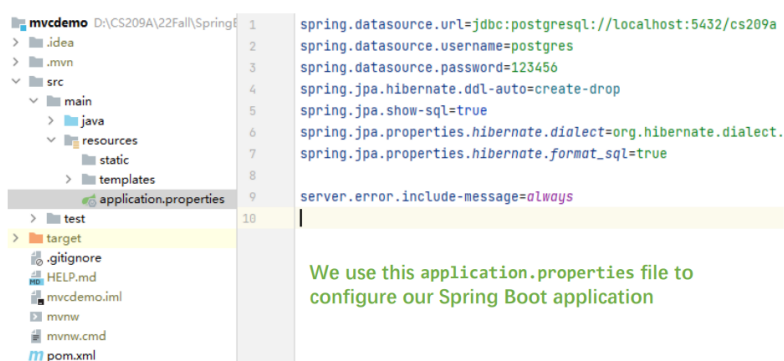
- 一个特殊的起始（父）项目，用于初始化项目设置，包括：
 - 默认的 Maven 插件
 - 默认依赖及版本管理
 - 默认属性与配置

• spring-boot-starter-web

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- 会传递性地引入与 Web 开发相关的所有依赖。
- 该 starter 包含（示例）常见的库与子依赖（如 Jackson、Tomcat、Spring MVC 等），开发者无需逐一添加这些依赖。
- 在 pom.xml 中只需加入：
 - groupId: org.springframework.boot
 - artifactId: spring-boot-starter-web
- 结果：自动获得完整的 Web 开发运行时环境（序列化/反序列化、内嵌容器、MVC 支持等）。

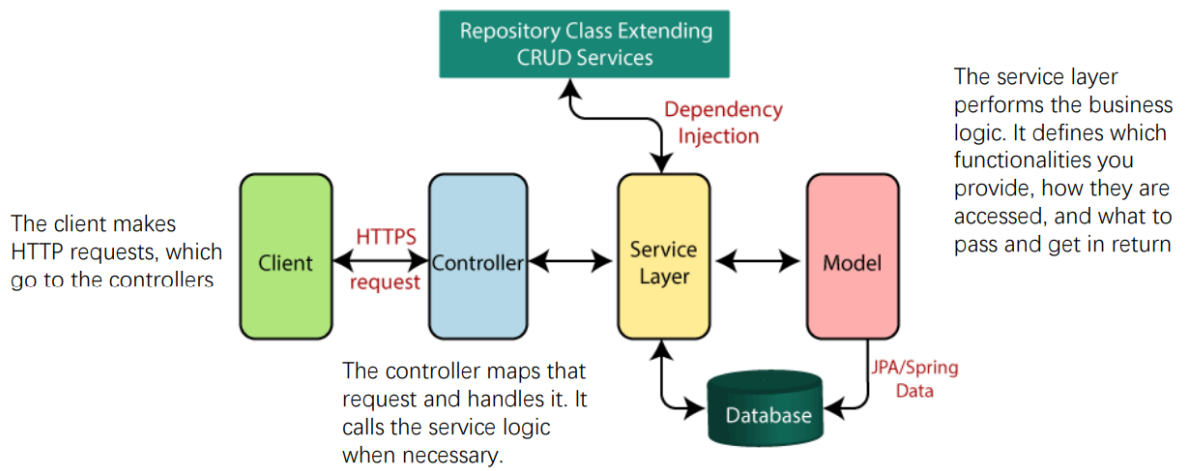
• 使用 application.properties 文件来配置 Spring Boot 应用程序。



```
1 spring.datasource.url=jdbc:postgresql://localhost:5432/cs209a
2 spring.datasource.username=postgres
3 spring.datasource.password=123456
4 spring.jpa.hibernate.ddl-auto=create-drop
5 spring.jpa.show-sql=true
6 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
7 spring.jpa.properties.hibernate.format_sql=true
8
9 server.error.include-message=always
10
```

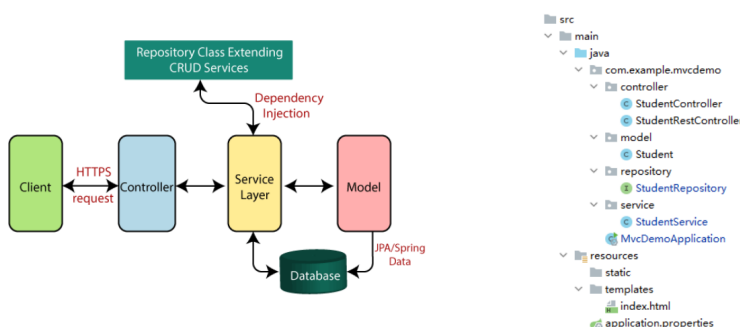
We use this application.properties file to configure our Spring Boot application

• Application Class

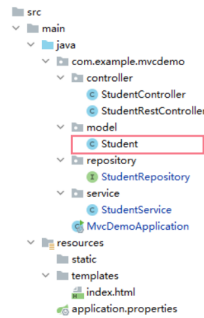


- 请求到控制器
 - 客户端发出 HTTP 请求，发送到控制器（Controller）。
 - 控制器将请求映射并处理必要的逻辑；在需要时调用服务层的业务逻辑。
 - 控制器与服务层之间通过依赖注入（Dependency Injection）使用仓库类（Repository）来扩展增删改查（CRUD）服务。
 - 服务层负责执行业务逻辑：定义提供哪些功能、如何访问这些功能、以及传入和返回的数据格式。
- 模型说明
 - 模型（Model）包含应用中的数据模型 / 实体 / 数据结构，例如用户、书籍、商品、账户等。
 - 模型对象通过 JPA / Spring Data 映射到数据库表。
 - 服务层与模型交互以执行业务计算，数据库操作（创建、读取、更新、删除）通常由仓库类完成，并注入到服务类中。
- 返回与数据操作
 - 最终返回给客户端的是网页（Web 应用）或 JSON 数据（REST 应用）。
 - 数据操作（例如创建、读取、更新、删除）通过仓库类执行，仓库类被注入到服务层以供使用。
 - 控制器负责将处理结果返回给客户端（页面或 JSON），整个请求流从客户端 → 控制器 → 服务层 → 仓库/数据库 → 返回结果。

A Simple Student Management Web Application



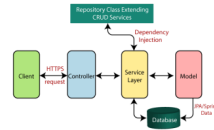
Model



```
Student
Student()
Student(String, String)
Student(Long, String, String)
getId(): Long
setId(Long): void
getName(): String
setName(String): void
getEmail(): String
setEmail(String): void
toString(): String | Object
id: Long
name: String
email: String
```

JavaBean: a POJO that conforms to certain conventions

- All properties are private
- Public setters and getters
- A public no-argument constructor



• JavaBean（符合约定的 POJO）的一般规范：

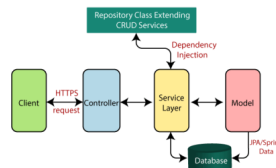
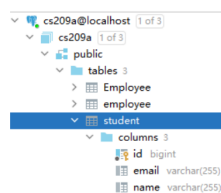
- 所有属性为私有（private）。
- 提供公共的 getter 与 setter 方法。
- 提供一个公共的无参构造器。

• 映射模型类到数据库表

Mapping Model Class to Database Table

- **@Entity**: specifies that the class is an entity and is mapped to a database table
- **@Table**: specifies the name of the database table to be used for mapping (default is the class name)
- **@Id**: specifies the primary key of an entity
- **@GeneratedValue**: specifies the generation strategies for the values of primary keys (default: auto).

```
@Entity
@Table
public class Student {
    4 usages
    @Id
    @GeneratedValue
    private Long id;
    5 usages
    private String name;
    5 usages
    private String email;
    1 usage -> yidatoo
    public Student() {
    }
}
```

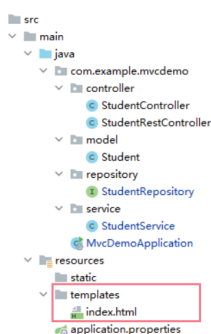


- **@Entity**: 指定该类是实体，并映射到数据库表。
- **@Table**: 指定用于映射的数据库表名（默认使用类名）。
- **@Id**: 指定实体的主键字段。
- **@GeneratedValue**: 指定主键值的生成策略（默认：auto）。
- 示例说明：Student 类包含私有字段 id、name、email，提供无参构造器；在数据库中对应 student 表及列（如 id、email、name）。

• Thymeleaf 模板

View

A Thymeleaf template that contain HTML code with Thymeleaf-specific syntax (th:text, th:each, etc.)



```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <title>Spring Boot Demo</title>
</head>
<body>
    <h1>Student List</h1>
    <table>
        <tr th:each="student: ${students}">
            <td th:text="${student.id}">ID</td>
            <td th:text="${student.name}">Name</td>
            <td th:text="${student.email}">Email</td>
        </tr>
    </table>
</body>
</html>
```

<table> will be replaced by values of the student variable when rendered

- 使用 Thymeleaf 模板编写 HTML，包含 Thymeleaf 特定语法（如 th:each、th:text 等）。
- 模板示例：使用 th:each 遍历 students 列表，使用 th:text 显示 student.id、{student.name}、\${student.email}。
- 渲染过程：模板中的占位符会被实际数据替换，生成最终要发送到浏览器的 HTML。

- @ Controller：将类标记为 Web 请求处理器（控制器）

Controller

@Controller is a class-level annotation that marks a class as a web request handler. It is mostly used with @RequestMapping annotation.

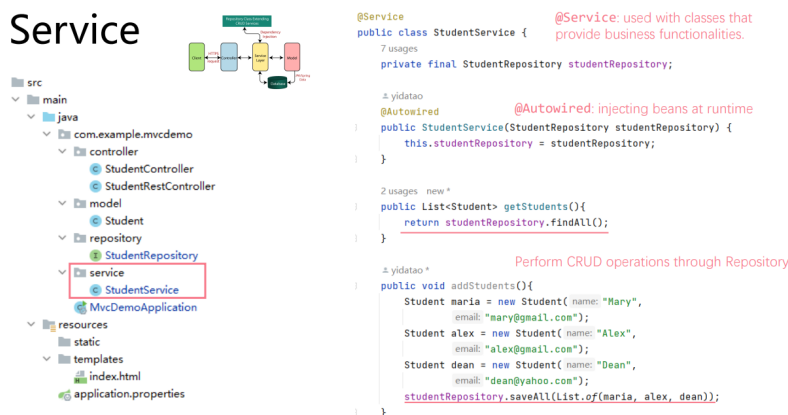


When a user accesses the /list URL, Thymeleaf will process the index.html template, replace the placeholders (e.g., \${students}) with actual values, and render the final HTML to be sent to the browser.

- 通过构造函数注入 Service（例如注入 StudentService）以调取业务方法。
- 使用请求映射（如 /list）处理客户端请求；在处理方法中通过 model.addAttribute("students", studentService.getStudents()) 将数据放入模型，然后返回视图名（例如 "index"）。
- 当用户访问对应 URL 时，Thymeleaf 处理模板并把替换后的 HTML 返回给客户端。

- @ Service：用于实现业务功能的类。

Service



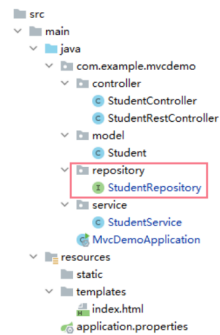
@Service: used with classes that provide business functionalities.

@Autowired: injecting beans at runtime

Perform CRUD operations through Repository

- 通过构造函数注入 Repository（或使用自动注入）以访问持久层。
- 提供业务方法，例如 getStudents() 调用 studentRepository.findAll() 返回所有记录。
- 提供数据写入方法，例如 addStudents() 创建若干 Student 实例并通过 studentRepository.saveAll(...) 保存到数据库。
- 服务层负责封装业务逻辑并通过仓库执行 CRUD 操作。

Repository



```
import com.example.mvcdemo.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
4 usages  △ yidatoo *
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

Entity/Model
class name

Type of the id

A **JpaRepository** interface defines basic methods for performing CRUD operations, sorting and paginating data.

- StudentRepository 接口继承自 JpaRepository<Student, Long>。
- 泛型参数说明：第一个为实体类名（Entity/Model），第二个为主键 ID 的类型。
- 通过继承 JpaRepository 即可获得对实体的基本 CRUD、排序和分页等操作方法，无需手动实现。
- JpaRepository 提供执行 CRUD 操作、排序和分页的基础方法。
- 开发者只需为每个域/实体（例如 Student）创建对应的 Repository 并继承 JpaRepository。
- 无需自己实现这些方法，Spring Data JPA 会自动生成实现（默认使用 Hibernate 作为实现）。
-

Keyword	Sample
And	findByLastnameAndFirstname
Or	findByLastnameOrFirstname
Is, Equals	findByFirstname, findByFirstnameIs, findByFirstnameEquals
Between	findByStartDateBetween
LessThan	findByAgeLessThan
LessThanEqual	findByAgeLessThanEqual
GreaterThan	findByAgeGreaterThan
GreaterThanEqual	findByAgeGreaterThanEqual
After	findByStartDateAfter
Before	findByStartDateBefore
IsNull	findByAgeIsNull
IsNotNull, NotNull	findByAge(Is)NotNull
Like	findByFirstnameLike
NotLike	findByFirstnameNotLike
StartingWith	findByFirstnameStartingWith
EndingWith	findByFirstnameEndingWith
Containing	findByFirstnameContaining
OrderBy	findByAgeOrderByLastnameDesc
Not	findByLastnameNot

```
4 usages  △ yidatoo *
public interface StudentRepository extends JpaRepository<Student, Long> {
    1 usage  new *
    List<Student> findByEmailLike(String email);
}
```

- 可以按照命名约定定义自定义查找方法，例如方法前缀应为：findBy、queryBy、countBy 等。
- 支持的关键字包括（示例）：And、Or、Is/Equals、Between、LessThan、GreaterThan、After、Before、IsNull、Like、NotLike、StartingWith、

EndingWith、Containing、OrderBy、Not 等。

- Spring 会根据方法名自动生成实现，无需手动编写具体 SQL/JPQL。

• Bootstrap — CommandLineRunner

```
@SpringBootApplication
public class MvcDemoApplication {
    // yidatoo
    public static void main(String[] args) {
        SpringApplication.run(MvcDemoApplication.class, args);
    }

    // yidatoo
    @Bean
    public CommandLineRunner commandLineRunner(StudentService service){
        return args -> {
            service.addStudents();
        };
    }
}
```

localhost:8080/list

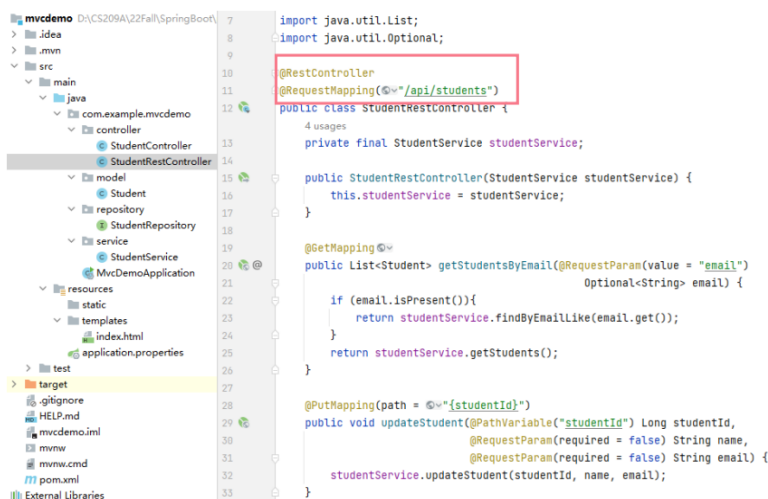
Student List

- 1 Mary mary@gmail.com
- 2 Alex alex@gmail.com
- 3 Dean dean@yahoo.com

- Spring Boot 的 CommandLineRunner 接口用于在应用初始化完成后只运行一次的一段代码（应用生命周期内执行一次）。
- 常见做法：在主类或配置类中声明一个 @Bean 返回 CommandLineRunner，通过注入 Service 在应用启动后执行初始化逻辑（例如填充测试数据）。
- 示例效果：启动时调用 service.addStudents(), 然后访问 /list 可看到预置的学生列表。

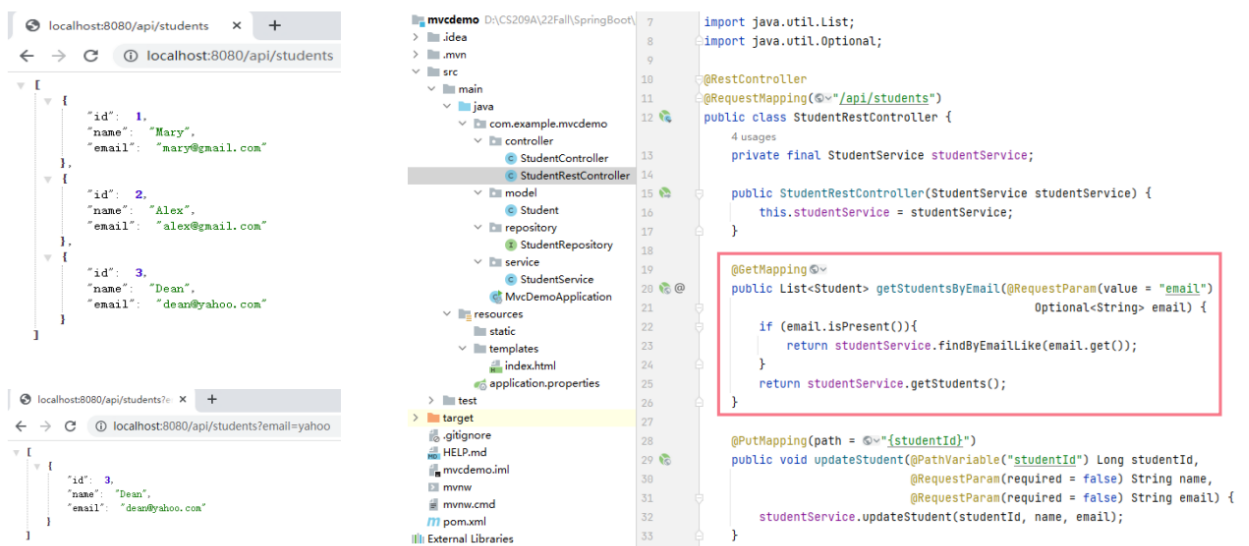
• 构建 RESTful Web 服务

- MVC 控制器与 RESTful 控制器的关键区别在于 HTTP 响应体的生成方式。
 - MVC 控制器：依赖视图技术返回 HTML 数据。
 - REST 控制器：直接以对象形式返回数据，作为 JSON 写入 HTTP 响应体。
- 使用 Spring Initializer 时，选择 Spring Web 即可满足构建 REST 服务的需求。

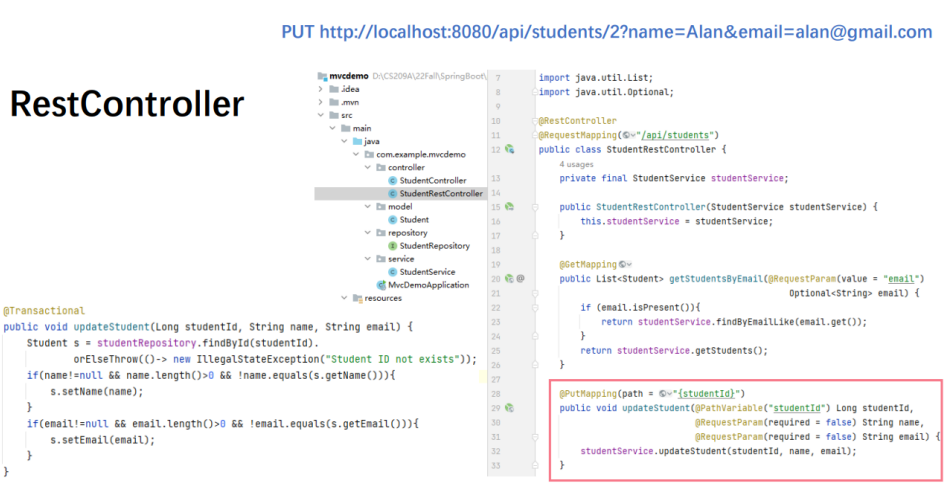


The screenshot shows an IDE with a project named 'mvcdemo'. The left sidebar shows the project structure with folders like 'main', 'resources', 'test', and 'target'. The 'main' folder contains 'com.example.mvcdemo', which includes 'controller', 'model', 'repository', 'service', and 'resources'. The 'controller' folder contains 'StudentController' and 'StudentRestController'. The 'StudentRestController' is selected, and its code is displayed in the editor. The code includes imports for 'List' and 'Optional', and annotations for '@RestController' and '@RequestMapping("/api/students")'. The class 'StudentRestController' has a private field 'studentService' and two methods: 'getStudentsByEmail' and 'updateStudent'.

- @RestController：将类标记为 REST 控制器，类中每个方法返回领域对象而不是视图（相当于 @Controller + @ResponseBody 的简写）。
- @RequestMapping：为该控制器中创建的所有 REST API 定义基础 URL（例如 @RequestMapping("/api/students")）。



- **@GetMapping**: 将 HTTP GET 请求映射到对应的方法，例如将 `/api/students` 的 GET 请求映射到某个处理方法。
- **@RequestParam**: 将查询字符串参数（如 `email`）的值绑定到方法参数中（例如 `Optional<String> email`），用于接收可选或必需的请求参数。



- **@PutMapping**: 将 HTTP PUT 请求映射到特定的处理方法（等同于 `@RequestMapping(method = RequestMethod.PUT)` 的简写）。
- **@PathVariable**: 从 URI 路径中提取值并绑定到方法参数，例如将路径中的 `{studentId}` 绑定到 `studentId` 参数。
- 示例用途：使用 PUT 更新指定 `studentId` 的学生信息，同时可通过 `@RequestParam` 接收可选的 `name`、`email` 等字段并传递给服务层进行更新。